

Тема 9. Проектиране на структури от данни. Ниво на абстракция и видимост. Обработка на йерархични данни.

Съдържание:

Проектиране на структури от данни.

Ниво на абстракция и видимост.

Обработка на йерархични данни.

Тема 9. Проектиране на структури от данни. Ниво на абстракция и видимост. Обработка на йерархични данни.

1. Мар. Определение: Мар в Elixir е асоциативна структура от данни, която позволява съхранение на ключ-стойност двойки. Подобно на хеш таблици в други езици, Мар предоставя удобен начин за управление и достъп до данни. Ключовете в Мар могат да бъдат от почти всеки тип (вкл. атоми, низове, числа и т.н.), стойностите също могат да бъдат от произволен тип. Тъй като ключовете не са ограничени, Мар е много гъвкав.

Създаване на Мар

Можете да създадете Мар по няколко начина:

Празен Мар: `empty_map = %{}`

Инициализиране с ключ и стойност:

`my_map = %{name: "Alice", age: 30}`

Създава списък от tuples и контейнер празен map:

`my_map = Enum.into([{:name, "Alice"}, {:age, 30}], %{})`

Създава списък от tuples: Параметърът `[{:name, "Alice"}, {:age, 30}]` е списък, съдържащ два tuple (или двойки), където всеки tuple е съставен от ключ и съответната стойност. В този случай, `{:name, "Alice"}` е ключ-стойност двойка, където `:name` е ключът, а `"Alice"` е стойността. Аналогично, `{:age, 30}` е ключ-стойност двойка с ключ `:age` и стойност `30`.

Използва Enum.into/2: Функцията `Enum.into/2` взема два аргумента: колекция (в случая списък от tuples) и контейнер (в случая `%{}`, който е празна мапа). Тя превръща елементите на първия аргумент (в този случай списъка от tuples) в ключ-стойност двойки в контейнера (празна мапа).

Тема 9. Проектиране на структури от данни. Ниво на абстракция и видимост. Обработка на йерархични данни.

Резултат: В резултат на изпълнението на израза, `my_map` ще има мап с ключове `:name` и `:age`, и съответстващите им стойности "Alice" и 30. Резултатът ще изглежда така: `my_map = %{name: "Alice", age: 30}`

С други думи, `Enum.into/2` може да се използва за преобразуване на списък от двойки (tuples) в мапа (map), което е удобно за създаване на структури от данни с ключ-стойност.

Достъп до стойности

Достъпът до стойности от Map може да се осъществи чрез оператор за създаване на списъци `[]` или `Map.get/2`.

```
my_map = %{name: "Alice", age: 30}
```

Пример с []:

```
name = my_map[:name]
```

```
# Извежда "Alice"
```

Пример с Map.get/2:

```
age = Map.get(my_map, :age)
```

```
# Извежда 30
```

Тема 9. Проектиране на структури от данни. Ниво на абстракция и видимост. Обработка на йерархични данни.

Актуализиране на Map

Актуализацията на стойности в Map не променя оригиналния Map, а вместо това създава нов с актуализираната стойност.

Пример:

```
updated_map = %{my_map | age: 31}
# updated_map е %{ "name" => "Alice", "age" => 31 }
```

Съществуват и функции в модула Map, като Map.put/3, които позволяват добавяне или обновяване на стойности.

Пример с Map.put/3:

```
my_map = Map.put(my_map, :gender, "female")
# my_map е %{ "name" => "Alice", "age" => 30, "gender" => "female" }
```

Изтриване на ключ-стойност

Можете да изтриете ключ чрез Map.delete/2.

Пример:

```
new_map = Map.delete(my_map, :age)
# new_map е %{ "name" => "Alice" }
```

Тема 9. Проектиране на структури от данни. Ниво на абстракция и видимост. Обработка на йерархични данни.

Употреба

Произволно асоциативно хранилище:

Map може да се използва за бързо съхранение на данни с произволни ключове, което е особено полезно при работа с JSON обекти.

```
user_data = %{name: "Alice", age: 30, city: "New York"}
```

Съставяне на настройки:

Може да се използва за съхраняване на конфигурационни параметри в приложение.

```
config = %{port: 4000, host: "localhost"}
```

Представяне на записи:

Използва се за представяне на записи в приложения, например записи на потребители, книги, и т.н.

```
book = %{title: "Elixir in Action", author: "Saša Jurić", year: 2019}
```

Изпълнение на функции за манипулация на данни:

Често Map-овете се използват в комбинация с функции, изброени в модула Enum, за обработка и манипулация на данни.

```
updated_books = Enum.map(books, fn book -> Map.put(book, :available, true) end)
```

Тема 9. Проектиране на структури от данни. Ниво на абстракция и видимост. Обработка на йерархични данни.

Кога да използвате Map:

- **Постоянна структура на данни:** Когато нуждата от динамични ключове е важна и когато работите с произволни набори от данни.
- **Четивност:** Когато искате да имате данни, представени с описателни ключове, които правят кода по-ясен и четим.
- **Динамика:** Когато искате да го сравнявате с други типове данни, тъй като Map е атрибутивен и поддържа динамично добавяне на нови ключ-стойности.
- **Лекота на работа:** Multithreading и паралелно програмиране, където Map може да бъде обработван от различни задачи, без опасност от модификация на оригиналните данни.

Обобщение за Map

Map в Elixir е мощен инструмент за работа с асоциативни структури от данни, предоставяйки бърз, ефективен и гъвкав начин за съхраняване и управление на данни в приложения. Независимо дали става дума за динамично генерирани данни, конфигурации или сложни записи, Map предлага полезни функции и синтаксис, който опростяват работата с данни.

Тема 9. Проектиране на структури от данни. Ниво на абстракция и видимост. Обработка на йерархични данни.

2. Struct. Определение: Struct в Elixir представлява специализиран подтип на Map, който позволява дефиниране на комплекси типове данни с предварително зададени ключове (или полета). Структурите предлагат по-строга структура и предсказуемост в сравнение с Map, като подобряват читаемостта и поддръжката на кода. Структурите са строго дефинирани: всяка структура има предварително определен набор от полета, което ограничева какви ключове могат да бъдат използвани. Структурите автоматично получават поле `__struct__`, което указва техния тип и предоставя на системата информация за използваната структура.

Създаване на Struct

Структурите се определят в модули чрез използване на `defstruct`. Всеки атрибут на структурата може да получи стойност по подразбиране.

Пример за дефиниране:

Структурите се дефинират в модули съгласно следния синтаксис:

```
defmodule MyStruct do
  defstruct field1: default_value1, field2: default_value2
end
```

```
defmodule User do
  defstruct name: "", age: 0
end
```

Структурата `User` е дефинирана с две полета: `name` и `age`, с подразбиращи се стойности — празен низ и 0.

Тема 9. Проектиране на структури от данни. Ниво на абстракция и видимост. Обработка на йерархични данни.

Конфигурации:

```
defmodule Config do
  defstruct database: "", host: "", port: 5432
end

config = %Config{database: "my_db", host: "localhost"}
```

Сложни структури:

```
defmodule Order do
  defstruct user: %User{}, items: [], total: 0
end

order = %Order{user: %User{name: "Alice"}, items: ["item1", "item2"], total: 100}
```

Анализ

```
defmodule Order do                                # Деклариране на модул с име Order.
  defstruct user: %User{}, items: [], total: 0    # Дефиниране на структура Order с три полета:
  # 1. user: по подразбиране е инстанция на структурата User (очаква се да бъде дефинирана в друг
  # модул).
  # 2. items: по подразбиране е празен списък, който ще съдържа елементите от поръчката.
  # 3. total: по подразбиране е 0, което представлява общата стойност на поръчката.
end
```


Тема 9. Проектиране на структури от данни. Ниво на абстракция и видимост. Обработка на йерархични данни.

```
# Създаване на нова инстанция на структура Order.  
order = %Order{  
# Задава полето user на новата инстанция Order с нова инстанция на структура User, където name е "Alice".  
  user: %User{name: "Alice"},  
# Задава полето items на поръчката с два елемента: "item1" и "item2".  
  items: ["item1", "item2"],  
# Задава полето total на поръчката на 100, представляваща общата стойност на поръчката.  
  total: 100  
}
```

Декларация на модул:

- `defmodule Order` до създава нов модул с име `Order`. Модулите в Elixir служат за логическа организация на кода и делят функционалността на различни области.

Дефиниране на структура:

- `defstruct` дефинира структурата за модула `Order`. Това е начин да се съхраняват свързани данни.

Полетата на структурата включват:

- **user:** За да се съхранява информация за потребителя, който прави поръчката. По подразбиране, структурата е инициализирана с нова инстанция на структурата `User`. Това означава, че очакваме друга структура с име `User`, която е дефинирана другаде в кода.
- **items:** Празен списък, който по-късно може да съдържа списък на поръчаните артикули. Дефинирането му като празен списък в началото предлага гъвкавост да добавяме елементи по-късно.
- **total:** Общата стойност на поръчката, инициализирана на 0. Тази стойност ще се актуализира, за да отрази цената на поръчаните артикули.

Тема 9. Проектиране на структури от данни. Ниво на абстракция и видимост. Обработка на йерархични данни.

Създаване на инстанция на структурата:

В последния ред, `order = %Order{...}` създаваме нова инстанция на структурата `Order` и задаваме стойности на полетата, както следва:

- **user:** `%User{name: "Alice"}`: Създаваме нова инстанция на структурата `User` (предполагаме дефинирана някъде другаде), задавайки `name` на `"Alice"`.
- **items:** `["item1", "item2"]`: Задаваме списък от два артикула за поръчката.
- **total:** `100`: Задаваме общата сума на поръчката на `100`.

Къде е подходящо да се използват Struct?

- **В комплексни приложения:** Структурите са много полезни в проекти, където работите с много взаимосвързани данни, както в уеб приложения, системи за управление на бази данни и (API) Приложно-програмен интерфейс.
- **За моделиране на бизнес логика:** Те се използват за инкапсулация на информация и логика, свързана с конкретни бизнес концепции или обекти.
- **При работа с външни ресурси:** При интеграция с API, защитени данни и конфигурации, структурите дават ясна представа за полетата и типовете данни, необходими за работа.

Тема 9. Проектиране на структури от данни. Ниво на абстракция и видимост. Обработка на йерархични данни.

Пример с Мар: Информационна система за земеделски култури

Представете си, че искаме да създадем приложение, което да записва информация за различни култури, включително тяхното име, сезон на отглеждане, нужда от вода, добив на хектар, както и друга свързана информация.

Определяме информация за култури с Mars

```
crops = %{\n  wheat: %{\n    name: "Пшеница",\n    growing_season: "Пролет",\n    water_requirement: "400 mm",\n    yield_per_hectare: 3.0 # в тона\n  },\n  corn: %{\n    name: "Царевица",\n    growing_season: "Лято",\n    water_requirement: "600 mm",\n    yield_per_hectare: 8.5 # в тона\n  },\n  rice: %{\n    name: "Ориз",\n    growing_season: "Лято",\n    water_requirement: "1000 mm",\n    yield_per_hectare: 4.5 # в тона\n  }\n}
```

Функция за извеждане на информация за култура

```
defmodule Farm do
```

```
  def print_crop_info(crop_map) do
```

```
    for {_, crop_details} <- crop_map do
```

Деструктуриране на всеки ключ-стойност двойка в crop_map, където ключът е игнориран (заменен с _),

а стойността (детайлите на културата) се присвоява на променливата crop_details.

```
    IO.puts("Култура: #{crop_details.name}")
```

```
        IO.puts("Сезон на отглеждане: #{crop_details.growing_season}")
```

```
        IO.puts("Нужда от вода: #{crop_details.water_requirement}")
```

```
        IO.puts("Добив на хектар: #{crop_details.yield_per_hectare}
```

```
тона")
```

```
        IO.puts("-----")
```

```
    end
```

```
  end
```

```
end
```

Извеждане на информация за културите

```
Farm.print_crop_info(crops)
```

Тема 9. Проектиране на структури от данни. Ниво на абстракция и видимост. Обработка на йерархични данни.

```
# Определяме информация за култури с Map
crops = %{
  wheat: %{
    name: "Пшеница",
    growing_season: "Пролет",
    water_requirement: "400 mm",
    yield_per_hectare: 3.0 # в тона
  },
  corn: %{
    name: "Царевица",
    growing_season: "Лято",
    water_requirement: "600 mm",
    yield_per_hectare: 8.5 # в тона
  },
  rice: %{
    name: "Ориз",
    growing_season: "Лято",
    water_requirement: "1000 mm",
    yield_per_hectare: 4.5 # в тона
  }
}

# Функция за извеждане на информация за култура
defmodule Farm do
  def print_crop_info(crop_map) do
    for {_, crop_details} <- crop_map do
      IO.puts("Култура: #{crop_details.name}")
      IO.puts("Сезон на отглеждане: #{crop_details.growing_season}")
      IO.puts("Нужда от вода: #{crop_details.water_requirement}")
      IO.puts("Добив на хектар: #{crop_details.yield_per_hectare} тона")
      IO.puts("-----")
    end
  end
end

# Извеждане на информация за културите
Farm.print_crop_info(crops)
```

```
Култура: Пшеница
Сезон на отглеждане: Пролет
Нужда от вода: 400 mm
Добив на хектар: 3.0 тона
-----
Култура: Царевица
Сезон на отглеждане: Лято
Нужда от вода: 600 mm
Добив на хектар: 8.5 тона
-----
Култура: Ориз
Сезон на отглеждане: Лято
Нужда от вода: 1000 mm
Добив на хектар: 4.5 тона
-----
```

Анализ

- **Определяне на култури:** Създадохме Map с името crops, който съдържа ключове (като wheat, corn, rice), асоциирани с под-картите (Map), представляващи различни свойства на всяка култура.
- **Функция print crop info:** Тази функция приема Map с информация за културите и извежда информацията на конзолата. Използваме for за итерация през Map и достъпване на полетата на всяка култура.
- **Извикване на функцията:** В последния ред извикваме функцията, за да отпечатаме информацията за записаните култури.

Тема 9. Проектиране на структури от данни. Ниво на абстракция и видимост. Обработка на йерархични данни.

Подходящи ситуации за използване на Мар в земеделски приложения

- ***Управление на информация за култури:*** Подходящо е за съхраняване на данни за различни култури и техните характеристики, което улеснява управлението на информацията.
- ***Обработка на селскостопански данни:*** Може да се използва за обработване на данни от полеви проучвания или отчети, където за различни култури е необходима динамична структура.
- ***Анализ на добиви:*** Може да се използва за съхраняване на данни за прогнози и анализ на добиви и нужди от ресурси за вземане на решения.
- Този пример илюстрира как Мар може да бъде полезен в селското стопанство за систематизиране на информация и управление на данни.

Тема 9. Проектиране на структури от данни. Ниво на абстракция и видимост. Обработка на йерархични данни.

Пример със struct: Система за управление на полетите в Elixir.

В този пример ще дефинираме структура, наречена Flight, която ще съдържа информация за полета, като **номер** на полета, **авиокомпания**, **дестинация**, **статус** на полета (напр. активен, отменен) и времето на излитане и кацане.

Дефиниция на структурата Flight:

```
defmodule Flight do
  defstruct number: "", airline: "", destination: "", status: :scheduled, departure_time: nil, arrival_time: nil
end
```

Създаване на модула за управление на полетите:

```
defmodule FlightManager do
  def schedule_flight(number, airline, destination, departure_time, arrival_time) do
    %Flight{
      number: number,
      airline: airline,
      destination: destination,
      status: :scheduled,
      departure_time: departure_time,
      arrival_time: arrival_time
    }
  end
end
```

Тема 9. Проектиране на структури от данни. Ниво на абстракция и видимост. Обработка на йерархични данни.

```
# статус на полета (напр. активен, отменен)
def cancel_flight(%Flight{} = flight) do
  %{flight | status: :canceled}
end

def update_flight_status(%Flight{} = flight, new_status) do
  %{flight | status: new_status}
end

def display_flight_info(%Flight{} = flight) do
  IO.puts("Flight Number: #{flight.number}")
  IO.puts("Airline: #{flight.airline}")
  IO.puts("Destination: #{flight.destination}")
  IO.puts("Status: #{flight.status}")
  IO.puts("Departure Time: #{flight.departure_time}")
  IO.puts("Arrival Time: #{flight.arrival_time}")
  IO.puts("-----")
end
end
```

Тема 9. Проектиране на структури от данни. Ниво на абстракция и видимост. Обработка на йерархични данни.

Използване на структурата

Нека да видим как можем да използваме структурата Flight и модула FlightManager в реалния пример:

```
defmodule Main do
  def run do
    # Създаване на полет
    flight1 = FlightManager.schedule_flight("AA123", "American Airlines", "New York", ~U[2023-10-01 10:00:00Z],
    ~U[2023-10-01 14:00:00Z])
    flight2 = FlightManager.schedule_flight("DL456", "Delta Airlines", "Los Angeles", ~U[2023-10-01 11:00:00Z], ~U[2023-
    10-01 15:00:00Z])

    # Извеждане на информация за полетите
    FlightManager.display_flight_info(flight1)
    FlightManager.display_flight_info(flight2)

    # Актуализация и изтриване на полет
    flight1_canceled = FlightManager.cancel_flight(flight1)
    FlightManager.display_flight_info(flight1_canceled)

    # Обновяване на статус на полет
    updated_flight2 = FlightManager.update_flight_status(flight2, :departed)
    FlightManager.display_flight_info(updated_flight2)
  end
end

# Стартиране на пример
Main.run()
```


Тема 9. Проектиране на структури от данни. Ниво на абстракция и видимост. Обработка на йерархични данни.

Реализацията е свързана с определен начин на извеждане на дата и час

В Elixir, `~U[2023-10-01 10:00:00Z]` представлява **UTC** дата и час с помощта на `Literal DateTime` синтаксиса. Нека разгледаме всяка част от структурата, за да обясним какво означава:

`~U[2023-10-01 10:00:00Z]`

- **`~U`**: - Това е префикс, който указва, че следващата стойност е UTC datetime. `~U` обозначава "UTC" (**Coordinated Universal Time**), и показва, че времето е в UTC, а не в локално време.

`[2023-10-01 10:00:00Z]`

- **`2023-10-01`**: Представлява датата (1 октомври 2023 г.).
- **`10:00:00`**: Представлява часа (10 часа, 0 минути и 0 секунди).
- **`Z`**: Указва, че времето е в UTC. "Z" е кратка форма за "здрава" синхронизация (Zulu time), която означава, че времето е в желания стандарт.

Какво представлява `~U` в Elixir?

Literal DateTime: `~U[date_time]` е начин в Elixir да се зададе конкретен времеви момент, който е ориентиран към UTC. Когато работите с времеви данни и особено при приложения, свързани с времето, е важно да бъдете ясно дали времето е в UTC или локално време.

Тема 9. Проектиране на структури от данни. Ниво на абстракция и видимост. Обработка на йерархични данни.

Пример за използване

Когато използвате `~U` синтаксиса, Elixir автоматично преобразува текста в `DateTime` структура, която може да се използва по-нататък в кода.

Пример за работа с `~U`

```
datetime = ~U[2023-10-01 10:00:00Z]
```

```
IO.inspect(datetime)
```

```
# Ще изведе: ~U[2023-10-01 10:00:00Z]
```

Можете да извършите операции с дата и час

```
future_datetime = DateTime.add(datetime, 3600) # Добавя 1 час
```

```
IO.inspect(future_datetime)
```

```
# Ще извежда: ~U[2023-10-01 11:00:00Z]
```

Обобщение

Като цяло, `~U[2023-10-01 10:00:00Z]` е много удобен и ясен метод за работа с дати и часове в Elixir, особено в контекста на приложения, които работят с глобални времеви зони или синхронизация. Използването на UTC помага да се избегнат грешки, свързани с локалното време, когато чествате и планирате събития за международна аудитория.

Тема 9. Проектиране на структури от данни. Ниво на абстракция и видимост. Обработка на йерархични данни.

```
defmodule Flight do
  defstruct number: "", airline: "", destination: "", status: :scheduled, departure_time: ""
end

defmodule FlightManager do
  def schedule_flight(number, airline, destination, departure_time, arrival_time) do
    %Flight{
      number: number,
      airline: airline,
      destination: destination,
      status: :scheduled,
      departure_time: departure_time,
      arrival_time: arrival_time
    }
  end

  def cancel_flight(%Flight{} = flight) do
    %{flight | status: :canceled}
  end

  def update_flight_status(%Flight{} = flight, new_status) do
    %{flight | status: new_status}
  end

  def display_flight_info(%Flight{} = flight) do
    IO.puts("Flight Number: #{flight.number}")
    IO.puts("Airline: #{flight.airline}")
    IO.puts("Destination: #{flight.destination}")
    IO.puts("Status: #{flight.status}")
    IO.puts("Departure Time: #{flight.departure_time}")
    IO.puts("Arrival Time: #{flight.arrival_time}")
    IO.puts("-----")
  end
end

defmodule Main do
  def run do
    # Създаване на полет
    flight1 = FlightManager.schedule_flight("AA123", "American Airlines", "New York", ~U[2023-10-01T10:00:00Z], ~U[2023-10-01T14:00:00Z])
    flight2 = FlightManager.schedule_flight("DL456", "Delta Airlines", "Los Angeles", ~U[2023-10-01T11:00:00Z], ~U[2023-10-01T15:00:00Z])

    # Извеждане на информация за полетите
    FlightManager.display_flight_info(flight1)
    FlightManager.display_flight_info(flight2)

    # Актуализация и изтриване на полет
    flight1 canceled = FlightManager.cancel flight(flight1)
  end
end
```

```
Flight Number: AA123
Airline: American Airlines
Destination: New York
Status: scheduled
Departure Time: 2023-10-01 10:00:00Z
Arrival Time: 2023-10-01 14:00:00Z
-----
```

```
Flight Number: DL456
Airline: Delta Airlines
Destination: Los Angeles
Status: scheduled
Departure Time: 2023-10-01 11:00:00Z
Arrival Time: 2023-10-01 15:00:00Z
-----
```

```
Flight Number: AA123
Airline: American Airlines
Destination: New York
Status: canceled
Departure Time: 2023-10-01 10:00:00Z
Arrival Time: 2023-10-01 14:00:00Z
-----
```

```
Flight Number: DL456
Airline: Delta Airlines
Destination: Los Angeles
Status: departed
Departure Time: 2023-10-01 11:00:00Z
Arrival Time: 2023-10-01 15:00:00Z
-----
```

Тема 9. Проектиране на структури от данни. Ниво на абстракция и видимост. Обработка на йерархични данни.

Структура на полета: Дефинирахме структура `Flight`, която съдържа необходимата информация за всеки полет.

FlightManager модула: Създадохме модул, който предоставя действия за управление на полети, включително:

- **Планиране на нов полет**
- **Отмяна на полет**
- **Обновяване на статус на полет**
- **Извеждане на информация за полет**

Изпълнение: В модула `Main` демонстрирахме как можем да планираме полети, да извеждаме информация за тях, да отменяме и обновяваме статуси.

Обобщение

В този пример създадохме структура за управление на полети, илюстрирайки как `Elixir` може да бъде използван за моделиране на реални сценарии. Структурите и модулите за управление на полетите предоставят лесен начин за организиране на данни и операции, свързани с авиацията, което е изключително полезно в приложения за управление на полети.

Тема 9. Проектиране на структури от данни. Ниво на абстракция и видимост. Обработка на йерархични данни.

Пример съчетаващ както Map, така и Struct

Създаване на Map и Struct за посещение на център по медитация

В примера използваме Map и Struct, за да управляваме информация за посещения на център по медитация. В този контекст можем да дефинираме информация за медитационните сесии и участниците.

Пример: Център по медитация

Дефиниране на структурата Participant:

```
defmodule Participant do
  defstruct name: "", email: "", phone: "", registration_date: nil
end
```

Дефиниране на структурата MeditationSession:

```
defmodule MeditationSession do
  defstruct topic: "", duration: 0, date: nil, participants: []
end
```

Тема 9. Проектиране на структури от данни. Ниво на абстракция и видимост. Обработка на йерархични данни.

Създаване на модул за управление на медитационни сесии:

```
defmodule MeditationCenter do
  def register_participant(%MeditationSession{} = session, name, email, phone) do
    new_participant = %Participant{
      name: name,
      email: email,
      phone: phone,
      registration_date: DateTime.utc_now()
    }
    %{session | participants: [new_participant | session.participants]}
  end

  def display_session_info(%MeditationSession{} = session) do
    IO.puts("Meditation Session Topic: #{session.topic}")
    IO.puts("Duration: #{session.duration} minutes")
    IO.puts("Date: #{session.date}")
    IO.puts("Participants:")
    Enum.each(session.participants, fn participant ->
      IO.puts(" - Name: #{participant.name}, Email: #{participant.email}, Phone: #{participant.phone}")
    end)
    IO.puts("-----")
  end
end
```

Тема 9. Проектиране на структури от данни. Ниво на абстракция и видимост. Обработка на йерархични данни.

Създаване на основен модул за изпълнение на примера:

```
defmodule Main do
  def run do
    # Създаване на нова медитационна сесия
    session = %MeditationSession{
      topic: "Mindfulness Meditation",
      duration: 60,
      date: ~U[2023-10-01 18:00:00Z],
      participants: []
    }

    # Регистрация на участници
    session = MeditationCenter.register_participant(session, "Alice", "alice@example.com", "+123456789")
    session = MeditationCenter.register_participant(session, "Bob", "bob@example.com", "+987654321")

    # Извеждане на информация за сесията
    MeditationCenter.display_session_info(session)
  end
end

# Стартиране на пример
Main.run()
```

Тема 9. Проектиране на структури от данни. Ниво на абстракция и видимост. Обработка на йерархични данни.

Анализ

- **Структура на участник (Participant):** Дефинираме структура, която съдържа информация за участниците, включваща име, имейл, телефон и дата на регистрация.
- **Структура на медитационна сесия (MeditationSession):** Дефинираме структура, която съдържа информация за медитационната сесия, включително темата, продължителността, датата и списък с участници.

Модул MeditationCenter:

- **register_participant/4:** Функция, която регистрира нов участник за сесия, създавайки нов обект от Participant и обновявайки списъка на участниците в текущата медитационна сесия.
- **display_session_info/1:** Функция за извеждане на информация за медитационната сесия, включително темата, продължителността и информация за участниците.

Модул Main: В основния модул:

Създаваме нова медитационна сесия и регистрираме участници.

Накрая извеждаме информация за сесията, включително детайли за участниците.

Обобщение

В този пример комбинираме Map и Struct, за да управляваме информацията за медитационната сесия и участниците в център по медитация. Структурите предоставят удобен начин за съхраняване на данни с ясна структура, докато Map може да се използва за динамично добавяне и управление на участници и допълнителна информация. Тази комбинация пресява функционалността и удобството на Elixir при изграждане на приложения за управление на посещения.

Тема 9. Проектиране на структури от данни. Ниво на абстракция и видимост. Обработка на йерархични данни.

Задача 1: *Да се създаде структура Book с полета: isbn, author, title, description, price, genre. Да се създаде функция за редактиране на данни. Да се създаде нов обект на структурата. Да се направи Update на стойностите на структурата. Да се изведе резултата.*

```
defmodule Book do # Деклариране на модул с име Book.
```

```
  defstruct isbn: "9786192406202",
```

```
    #Дефинира структура с поле isbn и задаване на стойност по подразбиране.
```

```
    author: "Rudyard Kipling",
```

```
    # Поле author с подразбираща се стойност.
```

```
    title: "The Jungle Book",
```

```
    # Поле title ....
```

```
    description: "It is about the Jungle",
```

```
    # Поле description .....
```

```
    price: 22.41,
```

```
    # Поле price .....
```

```
    genre: "novel"
```

```
    # Поле genre ....
```

```
def callstruct() do    #Функция, която показва стойностите на полетата на структурата с  
    подразбиращи се стойности.
```

```
books = %Book{}
```

```
    # Създава нов екземпляр на структурата Book с подразбиращи се стойности.
```

```
IO.puts(books.isbn)    # Извежда стойността на полето isbn.
```

```
IO.puts(books.author)  # Извежда ....author.
```

```
IO.puts(books.title)   # Извежда ....title.
```

```
IO.puts(books.description) # Извежда ....description.
```

```
IO.puts(books.price)   # Извежда .....price.
```

```
IO.puts(books.genre)   # Извежда .....genre.
```

```
end
```

Тема 9. Проектиране на структури от данни. Ниво на абстракция и видимост. Обработка на йерархични данни.

```
def makeObj(a, b, c, d, e, f) do
```

Функция, която приема параметри за създаване на нова структура Book с персонализирани стойности.

```
books = %Book{isbn: a, author: b, title: c, description: d, price: e, genre: f}
```

Създава нов екземпляр на структурата с предоставените стойности.

```
IO.puts(books.isbn)           # Извежда стойността на полето isbn от  
новосъздадения екземпляр.
```

```
IO.puts(books.author)        # Извежда.....author .....
```

```
IO.puts(books.title)         # Извежда.....title .....
```

```
IO.puts(books.description)   # Извежда.....description .....
```

```
IO.puts(books.price)         # Извежда....price .....
```

```
IO.puts(books.genre)        # Извежда.....genre .....
```

Тема 9. Проектиране на структури от данни. Ниво на абстракция и видимост. Обработка на йерархични данни.

```
upd = %{           # Създава нов мап с обновените стойности на полетата.  
  books | isbn: "Update_ISBN",           # Актуализира полето isbn.  
    author: "Update_Author",           # .....author.  
    title: "Update_Title",           # ..... title.  
    description: "Update_Description", # ..... description.  
    price: 99.99,           # ..... price.  
    genre: "Update_Genre"           # ..... genre.  
}  
  
  IO.inspect(upd) # Извежда обновената структура с новите стойности за преглед.  
end  
end
```

Тема 9. Проектиране на структури от данни. Ниво на абстракция и видимост. Обработка на йерархични данни.

```
isbn = IO.gets(„Enter ISBN: ") |> String.trim()
author = IO.gets("Enter Author: ") |> String.trim()
title = IO.gets("Enter Title: ") |> String.trim()
description = IO.gets("Enter description: ") |> String.trim()
price = IO.gets("Enter price : ") |> String.trim() |> String.toFloat()
genre = IO.gets("Enter genre: ") |> String.trim()

IO.puts(Book.makeObj({isbn, author, title, description, price, genre}))
```

Тема 9. Проектиране на структури от данни. Ниво на абстракция и видимост. Обработка на йерархични данни.

Задача 2: Да се създаде структура *Employees* с полета: *firstname*, *middlename*, *surname*, *job*, *workingdays*, *vacationday*. Да се въведат 3-ма служителя. Да се изчисли месечната заплата *monthllysalary* на всеки от служителите. Да се сортират служителите по име и по заплата. Данните да се записват в текстов файл, с опция за допълване на нови служители.

```
defmodule Tech do # Деклариране на модул с име Tech.
```

```
  @enforce_keys [:firstname, :middlename, :surname, :job, :workingdays, :vacationdays]
```

```
  # Определяне на ключовете, които са задължителни при създаване на структурата; всеки от тях е необходим.
```

```
  defstruct @enforce_keys ++ [:monthllysalary]
```

```
  # Дефиниране на структура с задължителни полета и допълнителното поле monthllysalary.
```

```
  defmodule Employees do
```

```
    # Деклариране на подмодул Employees в рамките на модула Tech.
```

```
    def listofemployees() do
```

```
      # Функция, която генерира и записва списък с служители.
```

```
      employee1 = %Tech{
```

```
        # Създаване на първи служител с конкретни стойности за полетата.
```

```
        firstname: "Ivan",
```

```
        middlename: "Petrov",
```

```
        surname: "Iliev",
```

```
        job: "Front-end programmer",
```

```
        workingdays: 30,
```

```
        vacationdays : 3
```

```
    }
```

```
    employee2 = %Tech{
```

```
      # Създаване на втори служител с конкретни стойности за полетата.
```

```
      firstname: "Ilian",
```

```
      middlename: "Dimitrov",
```

```
      surname: "Todorov",
```

```
      job: "Database analytics",
```

```
      workingdays: 40,
```

```
      vacationdays : 4
```

```
    }
```

Тема 9. Проектиране на структури от данни. Ниво на абстракция и видимост. Обработка на йерархични данни.

```
employee3 = %Tech{  
    # Създаване на трети служител с конкретни стойности за полетата.  
    firstname: "Radoslava",  
    middlename: "Dragostinova",  
    surname: "Kostova",  
    job: "Back-end programmer",  
    workingdays: 60,  
    vacationdays : 6  
}  
employee1 = %{\br/>    # Актуализиране на полето monthllysalary на employee1, основавайки се на workingdays.  
    employee1  
    | monthllysalary: employee1.workingdays * 21 # Изчислява месечната заплата, умножавайки  
    работните дни по 21.  
}  
employee2 = %{\br/>    # Актуализиране на полето monthllysalary на employee2, основавайки се на workingdays.  
    employee2  
    | monthllysalary: employee2.workingdays * 21 # Изчислява месечната заплата.  
}  
employee3 = %{\br/>    # Актуализиране на полето monthllysalary на employee3, основавайки се на workingdays.  
    employee3  
    | monthllysalary: employee3.workingdays * 21 # Изчислява месечната заплата.  
}
```

Тема 9. Проектиране на структури от данни. Ниво на абстракция и видимост. Обработка на йерархични данни.

```
employees = [employee1, employee2, employee3]      # Създаване на списък от служителите.
employees =
  employees
  |> Enum.sort_by(&Map.fetch(&1, :name))              # Сортиране на списъка по името на служителя.
  |> Enum.sort_by(&Map.fetch(&1, :job))               # Сортиране на списъка по длъжността на служителя.

{:ok, file} = File.open("employees.txt", [:append]) # Отваря или създава файл за добавяне на данни (append
mode).
|
for x <- 0..2 do                                     # Цикъл, който преминава през индексите 0 до 2 (т.е. през трите служителя).
  IO.binwrite(
    file,
    "#{Enum.at(employees, x).firstname}             # Записва името на служителя в файла.
    #{Enum.at(employees, x).middlename}             # Записва средното име на служителя.
    #{Enum.at(employees, x).surname}                # Записва фамилията на служителя.
    #{Enum.at(employees, x).monthllysalary}          # Записва месечната заплата на служителя.
    #{Enum.at(employees, x).job}                    # Записва длъжността на служителя.
    #{Enum.at(employees, x).workingdays}            # Записва работните дни на служителя.
    #{Enum.at(employees, x).vacationday}\n"          # Записва дните на отпуск на служителя и нов ред.
  )
end
end
end
end
```

Тема 9. Проектиране на структури от данни. Ниво на абстракция и видимост. Обработка на йерархични данни.

Описание на структурата

Дадената програма реализира система за управление на данни за студенти с помощта на Elixir. Основната функционалност включва добавяне, изтриване, показване и актуализиране на информация за студентите в текстов файл. Програмата също така проверява валидността на данните, които потребителят въвежда.

Основни компоненти на програмата:

Структура на данните:

Определена е структура Student с атрибути:

- fname: Първо име
- mname: Средно име
- lname: Фамилно име
- fnumber: Факултетен номер (приблизително 8 цифри)
- mark: Средна оценка (decimal)

Тема 9. Проектиране на структури от данни. Ниво на абстракция и видимост. Обработка на йерархични данни.

Основно меню:

Метод `mainMenu()` представя основно меню, от което потребителят може да избира различни опции:

- Въвеждане на нов студент
- Изтриване на студент
- Показване на всички студенти
- Актуализиране на студент
- Изход от програмата

Добавяне на студент:

- Метод `enterStudent()` осъществява въвеждане на информация за нов студент с проверки за валидност на имената, факултетния номер и оценката. При успешен вход, информацията е добавена в текстов файл.

Показване на студенти:

- Метод `showAll()` чете данните от файла и извежда информацията за студентите в подреден вид.

Изтриване на студент:

- Метод `deletePick()` предоставя опция за изтриване на студент по факултетен номер. Ако студент с указан факултетен номер не съществува, програмата уведомява потребителя.

Тема 9. Проектиране на структури от данни. Ниво на абстракция и видимост. Обработка на йерархични данни.

Плюсове на програмата:

- Лесен за използване интерфейс.
- Удобна организирана структура на данните.
- Добри проверки за валидност на входните данни.
- Защитена от дублиране на факултетни номера.

Минуси на програмата:

- Използва файлове, което може да доведе до проблеми с конкуренцията при многобройни записи.
- Липса на графичен интерфейс или работа с база данни.
- Тези функции и архитектура правят програмата подходяща за малки приложения, свързани с управление на студентски данни, особено в учебни или демо контексти.

Тема 9. Проектиране на структури от данни. Ниво на абстракция и видимост. Обработка на йерархични данни.

Актуализиране на информация:

- Метод `updatePick()` предлага на потребителя различни опции за актуализиране на информацията за студент (факултетен номер, имена и оценка). След успешна идентификация на студента, информацията може да бъде променена.

Валидация на данни:

- Програмата включва множество проверки за валидност на данните, за да избегне неправилно въвеждане.

Работа с файлове:

- Handy функции като `writeToFile()` и `readFileIntoListObjects()` управляват записването и четенето на информация от текстовия файл.

Примерен работен процес:

- Програмата стартира и показва менюто с опции.
- Потребителят избира опция (например "1" - за добавяне на нов студент).
- Програмата иска информацията за студента и извършва необходимите проверки.

След успешното добавяне, програмата се връща в основното меню, готова за следваща интеракция.

Тема 9. Проектиране на структури от данни. Ниво на абстракция и видимост. Обработка на йерархични данни.

```
defmodule Student do
```

```
# Определяне на структура от данни за студент с начални стойности
```

```
defstruct fname: "None", mname: "None", lname: "None", fnumber: "00000000", mark: 0.00
```

```
# Основно меню за взаимодействие с потребителя
```

```
def mainMenu() do
```

```
  IO.puts("====Main Menu====")
```

```
  IO.puts(" 1. Enter new student")
```

```
  IO.puts(" 2. Delete student")
```

```
  IO.puts(" 3. Show existing students")
```

```
  IO.puts(" 4. Update Student")
```

```
  IO.puts(" 5. Exit")
```

```
  choice = IO.gets("Pick: ") |> String.trim()
```

Тема 9. Проектиране на структури от данни. Ниво на абстракция и видимост. Обработка на йерархични данни.

Проверка дали входът е валидна цифра

```
if String.match?(choice, ~r/^\d$/) do
  choice = String.to_integer(choice)

  case choice do
    1 -> Student.enterStudent()  # Вход на нов студент
    2 -> Student.deletePick()    # Изтриване на студент
    3 -> Student.showAll()       # Показване на всички студенти
    4 -> Student.updatePick()    # Актуализиране на информацията за студент
    5 -> IO.puts("Exit")        # Изход от програмата
    _ ->
      IO.puts("Operation does not exist. Please pick again.")
      Student.mainMenu()        # Повторение на менюто при невалиден вход
  end
else
  IO.puts("Enter a valid operation!")
  Student.mainMenu() # Повторение на менюто при невалиден вход
end
end
```

Тема 9. Проектиране на структури от данни. Ниво на абстракция и видимост. Обработка на йерархични данни.

```
# Проверка дали студент с определен факултетен номер (FN) вече съществува
```

```
def isPresent(fnomer) do
  list =
    File.stream!("C:\\STRUCTURA\\STUDENT.txt")
    |> Stream.map(&String.trim_trailing/1)
    |> Enum.to_list()
```

```
# Търсене на студент по FN
```

```
subject =
  Enum.find(list, fn x ->
    check = String.split(x, ~r{ }) |> Enum.at(3)
    check == fnomer
  end)

if subject == nil do
  true # Студентът не съществува
else
  IO.puts("There is already a student with said FN. Please try again.")
  Student.enterStudent() # Повикване на метода за въвеждане на студент
end

end
```

Тема 9. Проектиране на структури от данни. Ниво на абстракция и видимост. Обработка на йерархични данни.

```
# Функция за проверка за дублиращи факултетни номера по време на актуализация
```

```
def dupFN(fnomer) do
  list =
    File.stream!("C:\\STRUCTURA\\STUDENT.txt")
    |> Stream.map(&String.trim_trailing/1)
    |> Enum.to_list()
```

```
# Търсене на съществуващ студент с указан FN
```

```
subject =
  Enum.find(list, fn x ->
    check = String.split(x, ~r{ }) |> Enum.at(3)
    check == fnomer
  end)

if subject == nil do
  true # Студентът не съществува
else
  IO.puts("There is already a student with said FN. Please try again.")
  Student.updateFN() # Повикване на метода за актуализиране на FN
end
```

```
end
```

Тема 9. Проектиране на структури от данни. Ниво на абстракция и видимост. Обработка на йерархични данни.

```
# Записване на списък от студенти в текстовия файл
```

```
def writeToFile(list) do
```

```
  # Отваряне на файла
```

```
  File.open("C:\\STRUCTURA\\STUDENT.txt", [:write])
```

```
  # Записване на всеки студент в файла
```

```
  for item <- list do
```

```
    File.write!(
```

```
      "C:\\STRUCTURA\\STUDENT.txt",
```

```
      "#{item.fname} #{item.mname} #{item.lname} #{item.fnomer} #{item.mark}\n",
```

```
      [:append]
```

```
    )
```

```
  end
```

```
  File.close("C:\\STRUCTURA\\STUDENT.txt") # Затваряне на файла
```

```
end
```


Тема 9. Проектиране на структури от данни. Ниво на абстракция и видимост. Обработка на йерархични данни.

Четене на данни от файла и преобразуването им в списък от обекти Student

```
def readFileIntoListObjects() do
  list =
    File.stream!("C:\\STRUCTURA\\STUDENT.txt")
    |> Stream.map(&String.trim_trailing/1)
    |> Enum.to_list()
```

Преобразуване на данните в обекти Student

```
toList =
  Enum.map(list, fn x ->
    line = String.split(x, ~r{ })
    stud = %Student{
      fname: Enum.at(line, 0),
      mname: Enum.at(line, 1),
      lname: Enum.at(line, 2),
      fnomer: Enum.at(line, 3),
      mark: Enum.at(line, 4)
    }
    stud # Връща обекта Student
  end)
toList # Връща списъка от студенти
end
```

Тема 9. Проектиране на структури от данни. Ниво на абстракция и видимост. Обработка на йерархични данни.

```
# Функция за въвеждане на нов студент
```

```
def enterStudent() do
```

```
  # Въвеждане на информация от потребителя
```

```
  fname = IO.gets("Enter First Name: ") |> String.trim()
```

```
  mname = IO.gets("Enter Middle Name: ") |> String.trim()
```

```
  lname = IO.gets("Enter Last Name: ") |> String.trim()
```

```
  fnum = IO.gets("FN: ") |> String.trim()
```

```
  mark = IO.gets("Student AVG Mark: ") |> String.trim()
```

```
# Проверка за валидност на имената и FN
```

```
  if String.match?(fname, ~r(^[a-zA-Z]+$)) do
```

```
    if String.match?(mname, ~r(^[a-zA-Z]+$)) do
```

```
      if String.match?(lname, ~r(^[a-zA-Z]+$)) do
```

```
        if String.match?(fnum, ~r(^[0-9]+$)) and String.length(fnum) == 8 do
```

```
          if String.match?(mark, ~r/^\d+(\.\d{1,2})?$/ ) do
```

```
            # Проверка дали FNum вече е зает
```

```
            Student.isPresent(fnum)
```

```
# Преобразуване на марката в десетично число
```

```
  {mark, _} = Float.parse(mark)
```

Тема 9. Проектиране на структури от данни. Ниво на абстракция и видимост. Обработка на йерархични данни.

```
if mark <= 6.00 and mark >= 2.00 do
    # Създаване на нов обект Student
    student = %Student{
        fname: fname,
        mname: mname,
        lname: lname,
        fnoMER: fnoMER,
        mark: mark
    }
    # Записване на данните в файла
    File.write!(
        "C:\\STRUCTURA\\STUDENT.txt",
        "#{student.fname} #{student.mname} #{student.lname} #{student.fnoMER} #{student.mark}\\n",
        [:append]
    )
    Student.mainMenu() # Връщане в основното меню
else
    IO.puts("No such mark exists")
    Student.mainMenu()
end
```

Тема 9. Проектиране на структури от данни. Ниво на абстракция и видимост. Обработка на йерархични данни.

```
else
    IO.puts("Invalid Mark")
    Student.mainMenu()
end
else
    IO.puts("FNomer contains unrecognized characters or is too short/long")
    Student.mainMenu()
end
else
    IO.puts("Last Name contains unrecognized characters")
    Student.mainMenu()
end
else
    IO.puts("Middle Name contains unrecognized characters")
    Student.mainMenu()
end
else
    IO.puts("First Name contains unrecognized characters")
    Student.mainMenu()
end
end
```

Тема 9. Проектиране на структури от данни. Ниво на абстракция и видимост. Обработка на йерархични данни.

```
# Показва всички студенти
```

```
def showAll() do
```

```
  list = Student.readFileIntoListObjects() # Четене на студенти от файла
```

```
  sorted = Enum.sort_by(list, & &1.fnomer) # Сортиране по FNomer
```

```
# Извеждане на информацията за всеки студент
```

```
for item <- sorted do
```

```
  IO.puts("#{item.fnomer} #{item.fname} #{item.mname} #{item.lname} #{item.mark}")
```

```
end
```

```
Student.mainMenu() # Връщане в основното меню
```

```
end
```

Тема 9. Проектиране на структури от данни. Ниво на абстракция и видимост. Обработка на йерархични данни.

Избиране на критерий за изтриване на студент

```
def deletePick() do
  IO.puts("====Delete by which criteria?====")
  IO.puts(" 1. FNomer")
  IO.puts(" 2. Back")
  pick = IO.gets("Criteria: ") |> String.trim()
  if String.match?(pick, ~r/^\d$/) do
    pick = String.to_integer(pick)
    case pick do
      1 -> Student.delByFNomer() # Изтриване по факултетен номер
      2 -> Student.mainMenu()   # Връщане в основното меню
      _ ->
        IO.puts("Non existing operation! Please try again")
        Student.deletePick() # Повторение на опита за изтриване
    end
  else
    IO.puts("Invalid Operation! Please try again.")
    Student.deletePick() # Повторение на опита за изтриване
  end
end
```

Тема 9. Проектиране на структури от данни. Ниво на абстракция и видимост. Обработка на йерархични данни.

```
# Функция за изтриване на студент по факултетен номер
```

```
def delByFNomer() do
```

```
  fnomer = IO.gets("Enter FNomer: ") |> String.trim()
```

```
  # Проверка за валидност на FN
```

```
  if String.match?(fnomer, ~r(^[0-9]+$)) and String.length(fnomer) == 8 do
```

```
    list =
```

```
      File.stream!("C:\\STRUCTURA\\STUDENT.txt")
```

```
      |> Stream.map(&String.trim_trailing/1)
```

```
      |> Enum.to_list()
```

```
  # Търсене на съществуващ студент по FN
```

```
  subject =
```

```
    Enum.find(list, fn x ->
```

```
      check = String.split(x, ~r{ }) |> Enum.at(3)
```

```
      check == fnomer
```

```
    end)
```

```
  # Извеждане на информацията за студент, който ще бъде изтрит
```

```
  IO.puts("You have deleted: ")
```

```
  IO.puts(subject)
```

```
  # Изтриване на студента от списъка
```

```
  list = List.delete(list, subject)
```

Тема 9. Проектиране на структури от данни. Ниво на абстракция и видимост. Обработка на йерархични данни.

```
toList =
  Enum.map(list, fn x ->
    line = String.split(x, ~r{ })
    stud = %Student{
      fname: Enum.at(line, 0),
      mname: Enum.at(line, 1),
      lname: Enum.at(line, 2),
      fnomer: Enum.at(line, 3),
      mark: Enum.at(line, 4)
    }
    stud # Връща обекта Student
  end)
  Student.writeToFile(toList) # Записване на оставащите студенти в файла
else
  IO.puts("Invalid FNomer! Try again.")
  Student.delByFNomer() # Повторение на опита за изтриване
end

Student.mainMenu() # Връщане в основното меню
end
```


Тема 9. Проектиране на структури от данни. Ниво на абстракция и видимост. Обработка на йерархични данни.

```
toList =
  Enum.map(list, fn x ->
    line = String.split(x, ~r{ })
    stud = %Student{
      fname: Enum.at(line, 0),
      mname: Enum.at(line, 1),
      lname: Enum.at(line, 2),
      fnomer: Enum.at(line, 3),
      mark: Enum.at(line, 4)
    }
    stud # Връща обекта Student
  end)
  Student.writeToFile(toList) # Записване на оставащите студенти в файла
else
  IO.puts("Invalid FNomer! Try again.")
  Student.delByFNomer() # Повторение на опита за изтриване
end

Student.mainMenu() # Връщане в основното меню
end
```

Тема 9. Проектиране на структури от данни. Ниво на абстракция и видимост. Обработка на йерархични данни.

Избиране на какво да се актуализира

```
def updatePick() do
  IO.puts("====What do you wish to update?====")
  IO.puts(" 1. FNomer")
  IO.puts(" 2. First Name")
  IO.puts(" 3. Middle Name")
  IO.puts(" 4. Last Name")
  IO.puts(" 5. Mark")
  IO.puts(" 6. Back")
  pick = IO.gets("Criteria: ") |> String.trim()
  if String.match?(pick, ~r/^\d$/) do
    pick = String.to_integer(pick)
    case pick do
      1 -> Student.updateFN()      # Актуализиране на FNomer
      2 -> Student.updateFName()   # Актуализиране на Първото име
      3 -> Student.updateMName()   # Актуализиране на Средното име
      4 -> Student.updateLName()   # Актуализиране на Фамилното име
      5 -> Student.updateMark()    # Актуализиране на Оценката
      6 -> Student.mainMenu()      # Връщане в основното меню
      _ ->
        IO.puts("Non existing operation! Please try again")
    end
  end
end
```

Тема 9. Проектиране на структури от данни. Ниво на абстракция и видимост. Обработка на йерархични данни.

```
else
  IO.puts("Invalid Operation! Please try again.")
  Student.updatePick() # Повторение на опита за актуализиране
end
end

# Актуализиране на оценката на студента
def updateMark() do
  fnomer = IO.gets("Enter Student Fnomer: ") |> String.trim()
  mark = IO.gets("Enter New Mark: ") |> String.trim()

  # Проверка за валидност на FN и оценката
  if String.match?(fnomer, ~r(^([0-9]+)$)) and String.length(fnomer) == 8 do
    if String.match?(mark, ~r(/^\d+(\.\d{1,2})?$/)) do
      list = Student.readFileIntoListObjects() # Четене на всички студенти
      {mark, _} = Float.parse(mark) # Преобразуване на оценката в десетично число

      # Актуализиране на номера студент
      upd =
        Enum.map(list, fn
          %Student{fnomer: ^fnomer} = x -> # Ако FNomer съвпада
            %Student{x | mark: mark}
        x ->
          x # Оставяне на другите студенти непроменени
        end)
```

Тема 9. Проектиране на структури от данни. Ниво на абстракция и видимост. Обработка на йерархични данни.

```
Student.writeToFile(upd) # Записване на актуализираните данни
    IO.puts("Mark has been Updated!")
else
    IO.puts("Unrecognized characters detected in Mark. Try again.")
    Student.updateMark() # Повторение на опита за актуализиране на оценката
end
else
    IO.puts("Invalid FNomer! Try again.")
    Student.updateMark() # Повторение на опита за актуализиране на оценката
end

Student.mainMenu() # Връщане в основното меню
end
```

Функция за актуализиране на първото име

```
def updateFName() do
    fnomer = IO.gets("Enter Student Fnomer: ") |> String.trim()
    new_name = IO.gets("Enter Replacement: ") |> String.trim()

    if String.match?(fnomer, ~r(^([0-9]+$)) and String.length(fnomer) == 8 do
        if String.match?(new_name, ~r(^([a-zA-Z]+$)) do
            list = Student.readFileIntoListObjects() # Четене на всички студенти
```

Тема 9. Проектиране на структури от данни. Ниво на абстракция и видимост. Обработка на йерархични данни.

Актуализиране на целевия студент

```
upd =
  Enum.map(list, fn
    %Student{fnomer: ^fnomer} = x -> # Ако FNomer съвпада
      %Student{x | fname: new_name}
    x ->
      x # Оставяне на другите студенти непроменени
  end)
Student.writeToFile(upd) # Записване на актуализираните данни
IO.puts("Name has been Updated!")
else
  IO.puts("Unrecognized characters detected in Name. Try again.")
  Student.updateFName() # Повторение на опита за актуализиране на името
end
else
  IO.puts("Invalid New FNomer! Try again.")
  Student.updateFName() # Повторение на опита за актуализиране на името
end

Student.mainMenu() # Връщане в основното меню
end
```

Тема 9. Проектиране на структури от данни. Ниво на абстракция и видимост. Обработка на йерархични данни.

Функция за актуализиране на средното име

```
def updateMName() do
  fnomer = IO.gets("Enter Student Fnomer: ") |> String.trim()
  new_name = IO.gets("Enter Replacement: ") |> String.trim()
  if String.match?(fnomer, ~r(^([0-9]+$)) and String.length(fnomer) == 8 do
    if String.match?(new_name, ~r(^([a-zA-Z]+$)) do
      list = Student.readFileIntoListObjects() # Четене на всички студенти
      # Актуализиране на целевия студент
      upd =
        Enum.map(list, fn
          %Student{fnomer: ^fnomer} = x -> # Ако FNomer съвпада
            %Student{x | mname: new_name}
          x ->
            x # Оставяне на другите студенти непроменени
        end)
      Student.writeToFile(upd) # Записване на актуализираните данни
      IO.puts("Name has been Updated!")
    else
      IO.puts("Unrecognized characters detected in Name. Try again.")
      Student.updateMName() # Повторение на опита за актуализиране на името
    end
  else
    IO.puts("Invalid New FNomer! Try again.")
    Student.updateMName() # Повторение на опита за актуализиране на името
  end
end
```

Тема 9. Проектиране на структури от данни. Ниво на абстракция и видимост. Обработка на йерархични данни.

```
Student.mainMenu() # Връщане в основното меню
end

# Функция за актуализиране на фамилното име
def updateLName() do
  fnomer = IO.gets("Enter Student Fnomer: ") |> String.trim()
  new_name = IO.gets("Enter Replacement: ") |> String.trim()
  if String.match?(fnomer, ~r(^([0-9]+$)) and String.length(fnomer) == 8 do
    if String.match?(new_name, ~r(^([a-zA-Z]+$)) do
      list = Student.readFileIntoListObjects() # Четене на всички студенти
      # Актуализиране на целевия студент
      upd =
        Enum.map(list, fn
          %Student{fnomer: ^fnomer} = x -> # Ако FNomer съвпада
            %Student{x | lname: new_name}
          x ->
            x # Оставяне на другите студенти непроменени
        end)
      Student.writeToFile(upd) # Записване на актуализираните данни
      IO.puts("Name has been Updated!")
    else
      IO.puts("Unrecognized characters detected in Name. Try again.")
      Student.updateLName() # Повторение на опита за актуализиране на името
    end
  else
    IO.puts("Invalid FNomer! Try again.")
    Student.updateLName() # Повторение на опита за актуализиране на името
  end
  Student.mainMenu() # Връщане в основното меню
end
```

Тема 9. Проектиране на структури от данни. Ниво на абстракция и видимост. Обработка на йерархични данни.

```
# Функция за актуализиране на факултетния номер
```

```
def updateFN() do
```

```
  old_fn = IO.gets("Enter Old FNomer: ") |> String.trim()
```

```
  new_fn = IO.gets("Enter New FNomer: ") |> String.trim()
```

```
# Проверка за валидност на стария и новия FN
```

```
if String.match?(old_fn, ~r(^([0-9]+$)) and String.length(old_fn) == 8 do
```

```
  if String.match?(new_fn, ~r(^([0-9]+$)) and String.length(new_fn) == 8 do
```

```
    # Проверка за дублиращи факултетни номера
```

```
    Student.dupFN(new_fn)
```

```
# Четене на всички студенти
```

```
list = Student.readFileIntoListObjects()
```

```
# Актуализиране на целевия студент
```

```
upd =
```

```
  Enum.map(list, fn
```

```
    %Student{fnomer: ^old_fn} = x -> # Ако старият FNomer съвпада
```

```
    %Student{x | fnomer: new_fn}
```

```
    x ->
```

```
    x # Оставяне на другите студенти непроменени
```

```
  end)
```

```
Student.writeToFile(upd) # Записване на актуализираните данни
```

```
IO.puts("FNomer has been Updated!")
```


Тема 9. Проектиране на структури от данни. Ниво на абстракция и видимост. Обработка на йерархични данни.

```
else
    IO.puts("Invalid New FNomer! Try again.")
    Student.updateFN() # Повторение на опита за актуализиране на FN
end
else
    IO.puts("Invalid Old FNomer! Try again.")
    Student.updateFN() # Повторение на опита за актуализиране на FN
end

Student.mainMenu() # Връщане в основното меню
end
end
```

Стартиране на основното меню при изпълнение на програмата

```
Student.mainMenu()
```

Обобщение

Тази програма е организирана по модулен начин, с ясно разделение на функциите, отговарящи за различни операции. Всеки метод се справя с конкретна задача, което улеснява поддръжката и разширението на кода. Важно е да забележим, че программната логика включва множество проверки, за да осигури коректността на въвежданите данни.

Тема 9. Проектиране на структури от данни. Ниво на абстракция и видимост. Обработка на йерархични данни.

Използвана литература:

[1] [https://elixir-lang.readthedocs.io/en/labooks /intro/15.html](https://elixir-lang.readthedocs.io/en/labooks/intro/15.html)